

Object oriented programming

Introduction to Programming

Rae Harbird, r.harbird@ucl.ac.uk

Object Oriented Programming (OOP)

So far we have used functions to organise code into small, manageable documented units. However, as programs become larger and more complex, it can become difficult to navigate all the possible pathways.



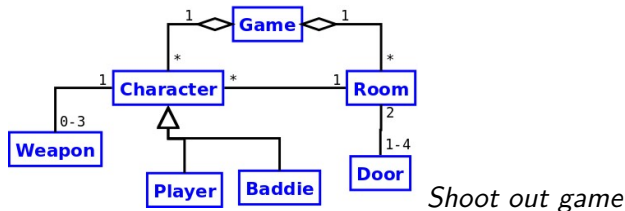
Figure 1: a bowl of spaghetti

Code that is difficult to read is sometimes called 'Spaghetti code'.

Note: Picture by Alessio Sbarbaro via Wikimedia

Object Oriented Programming

Object-oriented programming refers to a programming style that enables programmers to write better organised code that is easier to maintain and understand. “Full size” Python applications are commonly envisaged as collections of **interacting objects**.



Shoot out game

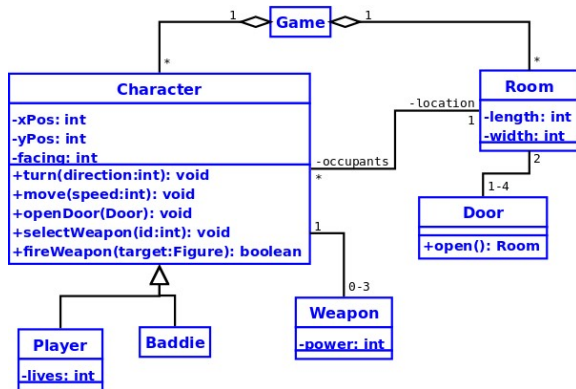
Each class corresponds to a set of objects (e.g the set of all the characters in the game) that all share a *common structure* and *behaviour*. The **class declaration** defines what the structure and behaviour is.

How to find classes

A programmer will design the application by thinking about the *things* (nouns) that would represent the application and these become objects in the program.

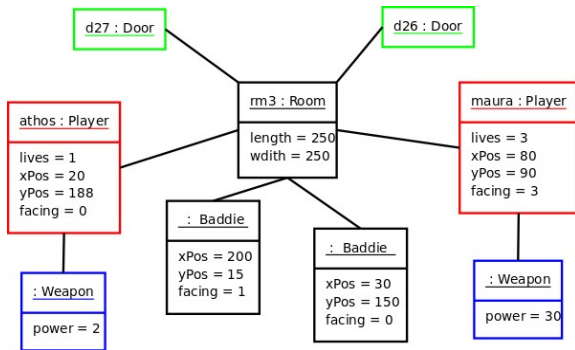
Attributes and actions

Each class has a set of *attributes* which determine its objects' structure, and a set of actions, called *methods*, which determine their behaviour.



Objects are instances

- ▶ Every **instance** (object) of the class has its own set of **instance variables** corresponding to the class attributes.
- ▶ The current values of an object's instance variables determine its state.



Class vs objects - what's the difference?

- ▶ A class is like a cookie cutter or template for a group of related data and a set of methods to manage that data:



Creating objects from a class

- ▶ You can create an **object** using the class which you can think of as being the biscuits created with the cutter:



- ▶ The objects behave in the way that the class says they can (using methods). The objects “know” about the information stored in their instance variables.

The structure of a class

In general, a class definition is of the following form:

Syntax `class` *ClassName* :
 `def` `__init__`(`self`, *parameterName*₁, *parameterName*₂, . . .) :
 constructor body

The special name `__init__` is used to define a constructor.

```
class BankAccount :  
    def __init__(self) :  
        self._balance = 0.0  
        . . .
```

A constructor defines and initializes the instance variables.

```
class BankAccount :  
    def __init__(self, initialBalance = 0.0) :  
        self._balance = initialBalance  
        . . .
```

There can be only one constructor per class. But a constructor can contain default arguments to provide alternate forms for creating objects.

The structure of a class (continued)

- ▶ The constructor is special method called `__init__()` which is used to initialise the **instance variables** in every object of the class.
- ▶ All methods contain the attribute `self` which refers to the object it is called on.
- ▶ Instance variables are prefixed by the word `self` which refers to the object.
- ▶ Instance variable names all start with an underscore `_`. This is a programming convention so that it's easy to tell which variables in the class are instance variables.

Improving the Pyramid

The pyramid program from the course unit about functions would be far more flexible if a “Pyramid” class were used to create Pyramid objects. Here is an extract from `src/pyramid.py`

```
class Pyramid :
```

```
    ## Constructs a pyramid with a given height from a given symbol.
    # @param theHeight the height of the pyramid (default = 2)
    def __init__(self, theHeight = 2, theSymbol = '*') :
        if theHeight < 2 :
            self._height = 2
        else :
            self._height = theHeight

        if len(theSymbol.strip()) == 0 :           # the symbol is empty
            self._symbol = '*'
        else :
            self._symbol = theSymbol.strip()[0]
```

Pyramid class continued

```
## Builds the spaces for a line
# @param currentLine the current line being printed.
# @returns a string containing the spaces for a line.
def spacesForPyramidLine(self, currentLine) :
    spacesForPyramidLine = ""
    spacesForPyramidLine = MARGIN +
                                (self._height + 1 - currentLine) * ' '
    return spacesForPyramidLine
```

This class definition is now a template that can be used to generate an unlimited number of pyramids, each with their own height and symbol. Take a look at `pyramid.py` and `pyramid_tester.py` in the examples folder.

Constructors

- ▶ A **constructor** is a special method that is called in order to create an object:

```
pyr1 = Pyramid(10, '&')
```

- ▶ Every call to the constructor generates a new instance of the class.
- ▶ A constructor always has the same name and does not return anything.
- ▶ The role of the constructor is usually to set the initial values of all instance variables. Your code should validate all the arguments passed in to the constructor and should provide default values for instance variables that haven't been specified.

Classes, constants and static variables

- ▶ Here is another example of a class. The `Coin` class can be used to create and manage `Coin` objects.
- ▶ Every object contains two **class-wide variables**, known as static variables.
- ▶ There is just one copy of the static variables for all the objects. Try using the `Coin` class and see how this works, every time a `Coin` object is created, either `headsSoFar` or `tailsSoFar` is incremented.

Coin class listing

```
import random

class Coin:
    # Static constants
    HEADS = 0
    TAILS = 1
    # Static variables
    headsSoFar = 0
    tailsSoFar = 0

    ## Constructor initializes the state of new Coin objects.
    def __init__(self):
        # set the attributes (the state in each Coin object)
        self.flip()

    ## Set the face of the coin to either heads or tails.
    def flip(self):
        self._face = random.randint(0,1)
        if self._face == Coin.HEADS:
            Coin.headsSoFar += 1
        else:
            Coin.tailsSoFar += 1
```

Coin class listing continued

```
# @return the number of heads so far
def get_heads_so_far():
    return Coin.headsSoFar

# @return the number of tails so far
def get_tails_so_far():
    return Coin.tailsSoFar

# @return a string representing the coin
def __str__(self):
    if self._face == Coin.HEADS:
        the_face = "H"
    if self._face == Coin.TAILS:
        the_face = "T"
    return the_face
```

Python Built-in Classes

You have already been using Python classes since the day that you started this course. The Python `Turtle` is a class, so are strings, lists, dictionaries and sets. Let's take a look at the `Turtle` class. You didn't use the `Turtle` library as if you were creating objects, but you can. Now that you know about classes you can create `Turtle` objects and call `Turtle` methods on the `Turtle` objects you have created.

The Python documentation tells us about what the `Turtle` class does and the methods available. The example program `square_turtles.py` in the `examples` folder creates two turtles and uses `Turtle` methods to draw a square.

Examples: Python Built-in Classes

Here are some examples, using the python interpreter, illustrating how the str class can be used to manipulate strings:

```
>>> name = input("Enter your name: ")
Enter your name: Rae
>>>
>>> name.upper()
'RAE'
>>> name.isalpha()
True
>>> name.isdigit()
False
>>>
```

More terminology

- ▶ The set of methods available in a class is known as the **public interface**. The class may contain other methods but these can be thought of as **private** and not available for public use.
- ▶ **Encapsulation** is the act of providing a public interface and hiding the implementation details. When you design your own classes, you will use encapsulation. That is, you will specify a set of public methods and hide the implementation details.
- ▶ Other programmers on your team can then use your classes without having to know about the implementation, just as you are able to make use of the methods in the `str` and `list` classes. Encapsulation enables changes in the implementation without affecting users of a class.

(p. 501), Horstmann, Cay S., Rance Necaie. Python for Everyone, Interactive Edition, 2nd Edition. Wiley.

Validating attributes

A well written constructor should validate all the instance variables like this example from the Pyramid class.

```
## Constructs a pyramid with a given height from a given symbol.
# @param theHeight the height of the pyramid (default = 2)
def __init__(self, theHeight = 2, theSymbol = '*') :
    if theHeight < 2 :
        self._height = 2
    elif theHeight > MAX_HEIGHT :
        self._height = MAX_HEIGHT
    else :
        self._height = theHeight

    if len(theSymbol.strip()) == 0 :
        self._symbol = DEFAULT_SYMBOL
    else :
        self._symbol = theSymbol.strip()[0]
```

Testing a Class

- ▶ Program coinprog.py is a tester program for the Coin class.
- ▶ To use the static methods get_heads_so_far() and get_tails_so_far(), we don't need to create an object.

```
# Name: coinprog.py
from coin import Coin

def main():

    for i in range(100) :
        coin = Coin()
        print(coin, end=", ")
    coin = Coin()
    print(coin)

    print("\n\nHeads so far: ", Coin.get_heads_so_far())
    print("Tails so far: ", Coin.get_tails_so_far())

    print("\nBye, Bye!\n")
```

Attribute getters and setters

- ▶ You might have wondered why the attributes in the classes we've shown so far have a prefix of an underscore: `_face` in `Coin` and `_height`, `_symbol` in `Pyramid`. The underscore is a programming convention used to indicate which variables are attributes.
- ▶ Attributes should **never** be accessed outside of the class they are defined in, they are private. Rather, the programmer should write methods to *get* the value of every attribute, called getters, and methods to *set* the value of every attribute, called setters.

Decorators

- ▶ In the next section, we will see that Python has some special techniques for managing the instance variables in classes.
- ▶ You can use this style of managing instance variables instead of writing your own getters and setters as shown previously.

Property decorator - @property

- ▶ Python provides a more formal way to manage the code associated with the attributes in a class using a **@property setter**.
- ▶ Let's take a look at the Pyramid class again rewritten using property setters. The code for this version of the Pyramid class is available in the examples folder in a module called `pyramid_with_properties.py`.

New style of constructor

- ▶ The constructor is much simpler, the validation has been moved elsewhere. We no longer need to use the underscore in the attribute name:

```
## Constructs a pyramid with a given height from a given symbol.  
# @param theHeight the height of the pyramid  
# @param theSymbol the symbol from which the pyramid should  
#           be made  
def __init__(self, theHeight = 2, theSymbol = '*') :  
    self.height = theHeight  
    self.symbol = theSymbol
```

New style of property getters and setters

- ▶ The getters and setters are defined using the `@property` decorator. You must use the format shown below when writing your own classes. Note that the getter and setter **do** use the attribute name with an underscore.
- ▶ You are probably wondering what is going on here. . . .
- ▶ The point is that the code in the getter and setter (and only that code) is responsible for managing the attribute.
- ▶ Elsewhere in this class, every time you change the value of an attribute or use the value of an attribute, these getters and setters are called without having to explicitly referred to them.

Example getters and setters

```
## The height property allows get/set access to the Pyramid's
# height value.
#
# The height must be between 2 and MAX_HEIGHT inclusive.
# If the value is < 2, height is set to 2.
# If the value > MAX_HEIGHT, height is set to MAX_HEIGHT
@property
def height(self):
    return self._height

## Sets the height attribute.
# @param value the height of the pyramid (default = 2)
@height.setter
def height(self, value):
    if value < 2 :
        self._height = 2
    elif value > Pyramid.MAX_HEIGHT :
        self._height = Pyramid.MAX_HEIGHT
    else :
        self._height = value
```

Making objects printable

- ▶ Python classes have a special method called `__str__()`. The purpose of `__str__()`, is to generate a string representing the object. We have seen some examples in the classes we've looked at so far.
- ▶ The `__str__()` method of the `Coin` class returns a string containing either an H or T:

```
## Return a string representation of a coin
# @return a string representing the coin
def __str__(self):
    if self._face == Coin.HEADS:
        the_face = "H"
    if self._face == Coin.TAILS:
        the_face = "T"
    return the_face
```

A printable Pyramid

- ▶ The `__str__()` method of the Pyramid class returns a string containing the pyramid:

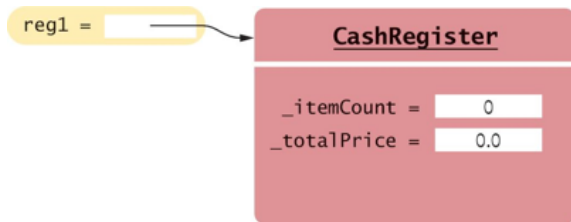
```
## Builds the pyramid line by line and returns an ascii
# representation.
# @returns pyramidString an ascii representation of the pyramid.
def __str__(self) :
    pyramidString = ""
    for i in range(1, self._height + 1) :
        pyramidString = pyramidString + self.pyramidLine(i)
    return pyramidString
```

Employee class

- ▶ The `Employee` class and associated tester program, `employee_prog.py` can be found in the examples folder. The attributes in the `Employee` class are managed using the `@property` decorator.
- ▶ An interesting point to note is that there is no input and output - no `print()` or `input()` statements in the `Employee` class. All the input and output happens in `employee_prog.py`. Can you make a guess as to why that might be?
- ▶ Well the answer is:
 - ▶ this style of writing a class makes it as widely useful as possible.
 - ▶ You could use the same `Employee` class whether the interface for the program is a Graphical User Interface (GUI) or a text-based interface or the program operates over the internet.
 - ▶ This is generally what you want.

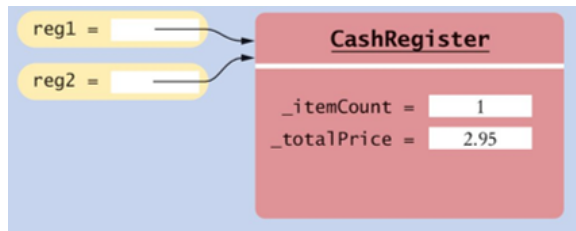
Object references

- ▶ When you store an object in a variable, the variable actually contains a *reference* to the object. You can see in the image below that the variable `reg1` contains a reference to a cash register object.



Object references continued

- ▶ When you copy a reference, by creating a new variable `reg2`, both the original and the new variable point to the same object:



- ▶ This means that variables containing objects can be used as arguments to functions and can be changed using the code in the function if necessary.

Note: diagrams from Python for Everyone, 2nd Ed., C. Horstmann and R. Necaie.

Object equivalence

- ▶ We are used to using the standard set of Python operators with the built-in data types that Python comes with (`==`, `!=`, `<=`, `>=`, `...`).
- ▶ Quite often, we will want to use these operators with the data types that we create ourselves.
- ▶ In this section we will look at how you can write code to determine what happens when you compare two objects. First of all we'll look at what happens when we don't do anything, we'll use the `Employee` class as an example.

Employee Equivalence

- ▶ In our simple `Employee` class an employee has a name and a salary.
- ▶ If we were comparing two employee objects, under what circumstances would we say that they are equal?
- ▶ Perhaps we might say that their names have to be the same and their salary also has to be the same. Here, I have created two `Employee` objects using the class in the examples folder using the Python interpreter:

Example

```
$ cd examples/  
$ python3  
Python 3.4.3 (default, Nov 28 2017, 16:41:13)  
[GCC 4.8.4] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> from employee_module import Employee  
>>> emp1 = Employee("Fiona")  
>>> emp2 = Employee("Fiona")  
>>> emp1 == emp2  
False  
>>>  
>>> id(emp1)  
140526591063488  
>>> id(emp2)  
140526591063824  
>>>
```

- ▶ Python thinks that these objects are not equal because it is comparing the id of the objects and, we can see, that those are not the same.

Comparing objects

- ▶ If you want to compare the attributes of the objects then you have to write an `__eq()` method. Here is the `__eq()` method for the `Employee` class:

```
def __eq__(self, rhsValue) :  
    if isinstance(rhsValue, Employee) :  
        return (self.name == rhsValue.name and  
                self.salary == rhsValue.salary)  
    else:  
        raise TypeError("Argument must be an Employee object.")
```

Other operators

Python includes a wide range of special methods that you can use to implement operators in any class such as:

Method	Description
<code>object.__lt__(self, other)</code>	returns True if object is less than the other object, False otherwise.
<code>object.__le__(self, other)</code>	returns True if object is less than or equal to the other object, False otherwise.
<code>object.__gt__(self, other)</code>	returns True if the object is greater than the other object, False otherwise.
<code>object.__ne__(self, other)</code>	returns True if the objects are not equal, False otherwise.
<code>object.__add__(self, other)</code>	returns the sum of the objects.
<code>object.__minus__(self, other)</code>	returns the value of the object minus the other object

A full listing can be found **here**

Storing objects in a list

- ▶ You can create an object and store the *reference* in a variable but what if you wanted to create and manage many objects?

```
employee1 = Employee("Fred Astair")  
employee2 = Employee("Ginger Rogers")  
employee3 = Employee("Toto the dog")
```

- ▶ But this has limitations. Alternatively, you could store the objects in a list and add a new object to the end of the list every time that an object is created.
- ▶ Take a look at `employeeprog_with_list.py`, try running it. In this program, any number of `Employee` objects can be created and added to a list.
- ▶ When the user types `v`, all the objects in the list are printed.

One object contains another object

- ▶ So far, the classes we have seen contain attributes which are simple data types: integers, floats, strings and so on.
- ▶ Attributes are not limited to these simple data types, they can be lists, dictionaries or even objects of types that you've created.
- ▶ This might not seem very intuitive and might be best illustrated with an example.

PetOwner and Pet classes

- ▶ Imagine a pet owner who could have zero or more pets. I could create a PetOwner class to represent the person owning the pets and I could create a Pet class to represent the pet. Take a look at the code in pet_owner_module.py.
- ▶ Since the PetOwner can have more than one pet, the Pet objects are stored in a list. The PetOwner class has an attribute which is a list of Pet objects.

```
## Constructs a PetOwner with a name and a list of pets.  
# @param owner_name the name of the pet owner.  
# @param pet_list the list of Pet objects  
#  
def __init__(self, the_name, the_pet_list):  
    self.name = the_name  
    self.pet_list = the_pet_list
```

- ▶ As you can see, a PetOwner has an attribute (instance variable) which is a list of pets.

Using the PetOwner and Pet classes

- ▶ The module `pet_owner_prog.py` contains a program which uses the `PetOwner` and the `Pet` class. Try running the program to see what it does.
- ▶ Now take a look at the setter for the `_pet_list` attribute, it becomes clearer what the list contains:

```
##  
# Check that the objects in the list are Pet objects.  
# It is valid for the list to be empty.  
@pet_list.setter  
def pet_list(self, the_pet_list):  
  
    for pet in the_pet_list:  
        if isinstance(pet, Pet) == False :  
            raise TypeError("List of pets contains an object which  
                             doesn't have type \"Pet\".")  
    self._pet_list = the_pet_list
```

Copying objects

- There are times when you will want to make a copy of an object. We know that just assigning one variable to another won't do it:

```
>>> from employee_module import Employee
>>> emp1 = Employee("Fiona")
>>> emp2 = Employee("Fiona")
>>> emp1 == emp2
False
>>>
>>> id(emp1)
140526591063488
>>> id(emp2)
140526591063824
>>>
```

Copying solution

- ▶ Instead, you should use the Python `deepcopy()` function:

```
>>> import copy
>>> from employee_module import Employee
>>> emp1 = Employee("Fiona")
>>> print(emp1)
Name: Fiona, salary: 10000
>>> emp2 = copy.deepcopy(emp1)
>>> print(emp2)
Name: Fiona, salary: 10000
```

- ▶ More detail on how the `deepcopy()` function works can be found in **Section 15.6, Think Python**